探索 GPU 加速向量检索：NVDIA Cagra 在微信大规模推荐系统中的应用实践

腾讯程序员 腾讯技术工程 2026年3月20日 02:36 广东 6人



作者: yessitkong (微信基础架构 AI Infra 团队)

引言

在当今的互联网服务架构中，向量检索技术已成为推荐系统、搜索引擎、内容匹配等核心业务场景的关键组件。随着深度学习模型的广泛应用，如何在海量向量数据中高效进行近似最近邻（Approximate Nearest Neighbor, ANN）搜索，直接影响着在线服务的用户体验和业务效果。

Cagra（CUDA Accelerated Graph-based Retrieval Algorithm）是 NVIDIA 推出的基于 GPU 加速的图索引 ANN 算法，也是 RAPIDS cuVS 库的核心组件之一。与过去业界广泛使用的传统 CPU-based ANN 算法（如 HNSW、IVF 等）相比，Cagra 充分利用了 GPU 的强大并行计算能力，在保持高召回率的同时，能够提供显著更高的吞吐量，满足不同业务对高性能、低成本的极致要求。

本文将分享我们团队如何攻克多项工程难题，**在业界率先将 Cagra GPU 图索引大规模应用于核心线上推荐业务**的技术实践与架构演进。

Cagra 算法原理与核心优势

核心技术原理

Cagra 采用基于图的索引结构，其核心思想是将向量数据构建成一个近似 k-近邻图（k-NN graph），然后在查询时通过启发式图遍历算法快速定位最近邻向量。

与传统的 HNSW 算法相比，Cagra 的结构设计针对 GPU 架构进行了深度定制，主要区别在于：

1. **单层图结构**：Cagra 采用单层图设计，摒弃了 HNSW 复杂的的多层分层结构，更利于显存的连续访问。
2. **固定出度**：Cagra 每个节点的出度固定，而 HNSW 的出度只需小于等于给定值，这使得 GPU 上的内存分配和线程调度更加规整。

3. **批量化检索**：HNSW 每次选取一个点遍历邻点并加入候选集；而 Cagra 每次会同时从候选集中选择 K 个未扩展的点进行并发扩展，然后统一更新候选集，极大地提升了并行度。

为了在 GPU 上实现更高的并行度与准确率，Cagra 在构建图时需要权衡两个关键指标：

- **全图连通性**：由于采用单层图设计且检索起始点选取较随机，必须确保所有节点双向连通，这是保证最终准确率的前提。
- **遍历效率优化**：在高维数据图的遍历中，CPU 侧通常依赖 hubs（高度连接的节点）来加速收敛。但在 GPU 场景下策略截然相反：GPU 采用分批处理机制，访问节点集合扩散得越快，越能发挥并行优势。因此，减少对 hubs 的重复遍历、让节点访问更加均匀分散，反而能加快查询收敛速度。

线上化改造与工程实践

由于 Cagra 诞生之初更侧重于离线大规模场景，为了将其适配到严苛的线上高并发服务中，我们与 NVIDIA 技术团队进行了深入的探讨，并对底层逻辑进行了大量优化。

1. 适应生产环境的建图优化

原论文采用 NNDescent 算法构建正向图，反转得到反向图后合并，再根据“2跳可达点数”对边进行排序和裁剪。然而在实际应用中，这种建图方式过度依赖大容量的 Pinned Memory（锁页内存），难以在标准的容器化生产环境中规模化部署。

为此，我们设计了**渐进式子图优化方案**： 首先随机构建连通图，通过多次小范围遍历筛选出子图，再对子图执行与论文类似的建图流程，逐步优化全图结构。最后辅以一轮连通性调整，在大幅降低内存依赖的同时，达到了与全图 NNDescent 相同的准确率效果。

2. 极致的性能表现

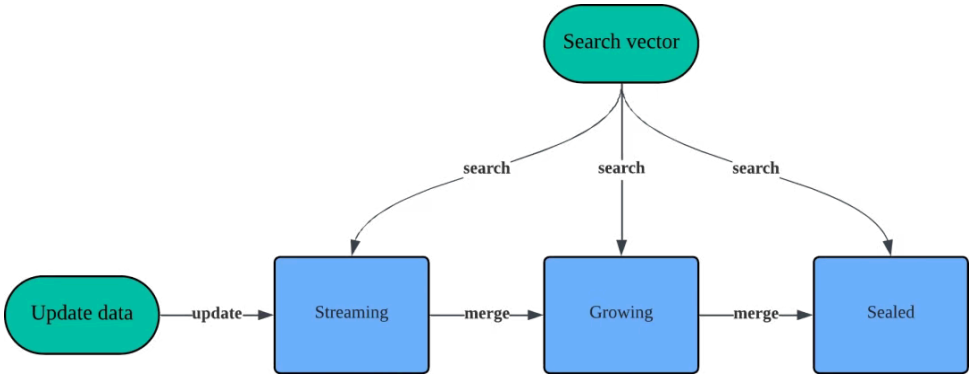
在我们的生产级数据集（千万级规模，高维向量）测试中，Cagra 展现出了惊人的性能优势：

- **建库性能**：在保证召回率 ≥ 0.9 的前提下，传统 HNSW 算法需要耗费数十分钟甚至小时级的时间，而 Cagra 将建索引时间**缩短至原来的 1/30 以下**（仅需几分钟）。
- **检索吞吐**：测试表明，当 Batch Size 增加时，Cagra 在同等硬件成本下可以获得远超 CPU 的 QPS。Batch 越大，QPS 提升越显著，完美契合高并发的推荐场景。

分布式向量检索系统的架构演进

为了将 GPU 检索能力无缝接入在线服务，我们的微信分布式向量检索系统 SimOL 采用了类似 LSM-Tree（Log-Structured Merge-Tree）的**三层分层存储架构**，巧妙平衡了数据时效性与 GPU 索引更新效率之间的矛盾。

三层架构设计



1. **Streaming 层（流式层）**
   - **定位**：最新数据的实时写入层。
   - **策略**：数据量小但更新极频繁。采用**暴力计算（Brute Force）+ 实时更新**。利用 GPU 加速暴力计算，新数据写入秒级可见，无需等待索引构建。
2. **Growing 层（增长层）**
   - **定位**：中间缓冲层，承接 Streaming 层达到阈值后的合并数据。
   - **策略**：数据量中等。采用 **Cagra 索引 + 批量重建**。
3. **Sealed 层（封存层）**
   - **定位**：历史数据的稳定存储层。
   - **策略**：数据量庞大，基本不再变化。同样采用 **Cagra 索引**，追求极致的查询性能。

平滑切换机制：对于 Growing 和 Sealed 层，我们利用 Cagra 强大的批量构建能力，在备用 GPU 显存上构建新索引，构建完成后进行原子切换（Double Buffer），确保线上服务零中断。

并发支持与 CPU/GPU 协同优化

在线上服务中，单纯提升 GPU 性能是不够的，必须平衡 CPU 与 GPU 的使用率，打破木桶效应。

1. Filter（过滤）方案的抉择与优化

业务中普遍存在复杂的属性过滤逻辑。虽然 Pre-filter（前置过滤）理论上召回率更高，但在 GPU 上会导致显存占用激增且时延波动大。

我们最终采用了 **Post-filter（后置过滤）** 方案：通过自动放大检索的 $topK$ 值，在 GPU 检索后交由 CPU 进行过滤。为了缓解 CPU 侧随机内存访问的开销，我们对 CPU 缓存进行了深度优化：

- 将参与 Filter 的数据分块（Chunk）存储。
- 优化数据载入顺序并实现内存预取（Prefetch），大幅提高 CPU Cache 命中率。 该优化将单机 QPS 相对提升了约 25%。

2. Batch 聚合与协程唤醒机制

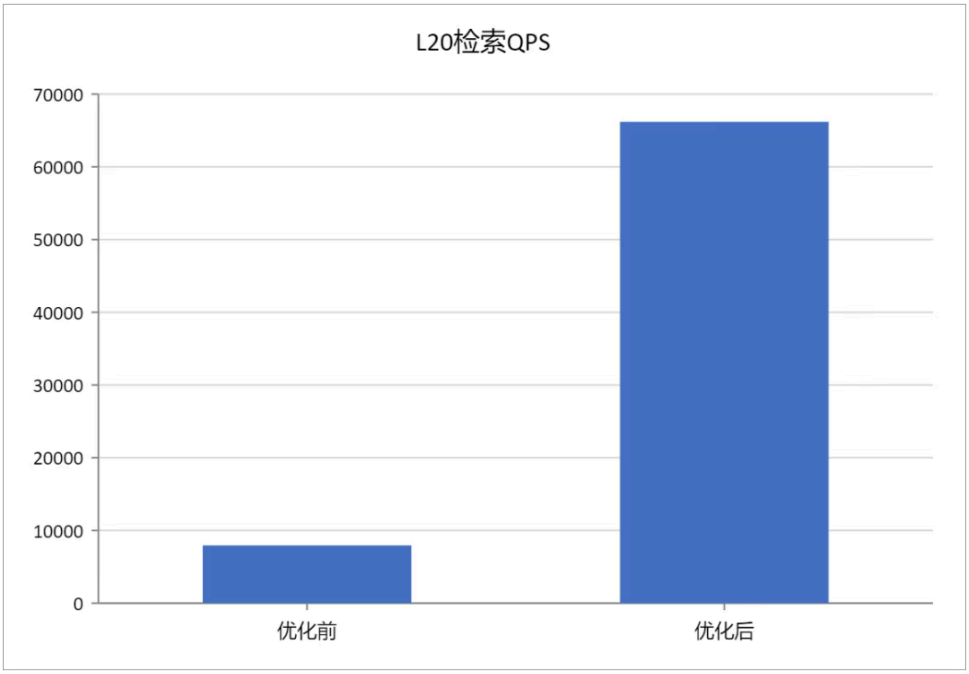
GPU 需要较大的 Batch 才能发挥算力，而 RPC 请求是离散的。如果采用传统的“一请求一线程”并在 Batch 完成时逐个跨线程唤醒，会导致极高的 CPU 上下文切换开销。

基于自研的协程框架，我们设计了**优化的聚合分发结构**：

- **聚合阶段**：各线程先独立聚合请求，再跨线程聚合成一个大 Batch 送入 GPU。
- **唤醒阶段**：Batch 完成后，采用“链式广播唤醒”机制，将跨线程通信次数从“总协程数量”降至“物理线程数量”。 这一改造使 CPU System 态的资源占用率大幅下降，单机吞吐量再次提升了 65%。

3. 聚合时延的精准把控

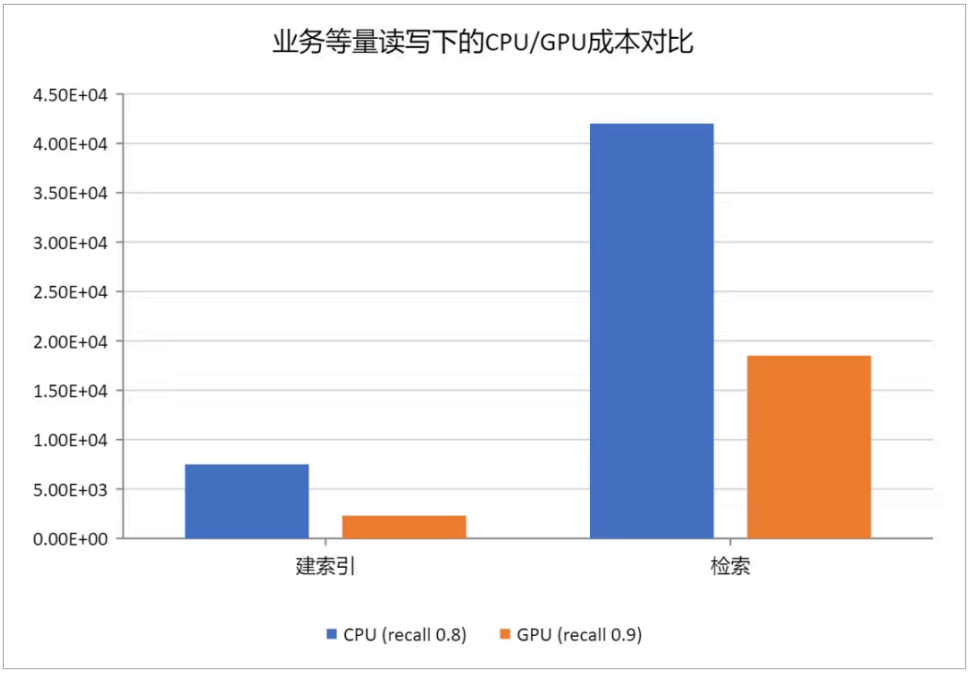
Batch 越大 GPU 利用率越高，但过长的等待会影响业务的 P99 延迟。经过海量线上请求的压测，我们找到了一个毫秒级的“黄金聚合窗口”，在不影响用户体验的前提下，最大化了 GPU 的吞吐收益。



业务落地效果

在微信视频号推荐系统的召回阶段，我们把向量检索召回从传统 CPU 架构升级到了基于 Cagra 的 GPU 架构，取得了显著的收益：

1. **突破性能瓶颈**：彻底解决了过去因 CPU 建库慢导致的数据合并拥堵问题，系统能够轻松应对极高并发的实时数据写入，系统的时效性提升了；
2. **召回率提升**：在更短的合并时延下，业务的整体召回率得到了显著提升，直接促进了推荐效果的增长；
3. **成本大幅下降**：
   - **检索成本**：在同等 QPS 且召回率更高的情况下，计算资源消耗**降低了 50% 以上**。
   - **建库成本**：得益于 GPU 极速的建图能力，索引构建成本**降低了约 70%**。



致谢

感谢 NVIDIA 技术团队在研发过程中提供的技术支持，感谢微信视频号推荐团队在架构升级过程中提供的协助。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 2960

留言

写留言

还没有留言，来写首评